

función.

3. Dados los diferentes valores de las acciones de YPF a lo largo del tiempo (que se representan como una secuencia de valores que representan el valor de las acciones cada día), se desea saber cuál es la mejor ganancia que se puede obtener al comprar acciones un día y venderlas otro día posterior.

Por ejemplo:

```
mejorGanancia(10, 7, 5, 8, 11, 9) = 6
mejorGanancia(10, 9, 8, 7) = 0
mejorGanancia(12) = 0
```

donde en la primer secuencia la mejor ganancia 6 se obtiene comprando a 5 y vendiendo a 11, en la segunda secuencia la mejor ganancia es 0 porque la mejor opción es no comprar ni vender, y en la tercera es 0 porque no hay suficientes días para comprar y vender.

Utilizando las operaciones del TAD secuencias, completar la definición de la función $\text{mejorGanancia} : \text{Seq Int} \rightarrow \text{Int}$ que dada una secuencia de valores calcula la mejor ganancia que se puede obtener, dando definiciones para reverse , op y b de manera que se verifique profundidad en $O(\lg n)$ y trabajo en $O(n)$, siendo n el largo de la secuencia.

```
mejorGanancia : Seq Int → Int
mejorGanancia s =
  let n = length s
      mejorVenta = reverse (fst (scan op b (reverse s)))
      diff = tabulate (λi → (nth i mejorVenta) - nth i s) (n - 1)
  in if n < 2 then 0 else reduce max 0 diff
```

3. Para realizar operaciones de compra y venta de dólares se definieron los siguientes tipos de datos:

```
type Pesos = Float
type Dolares = Float
type Cotizacion = Float
data Transaccion = Depositar Pesos | Comprar Int Cotizacion | Vender Int Cotizacion
```

donde $\text{Depositar } x$ representa la operación "depositar x pesos", $\text{Comprar } n \ c$, "comprar n dólares con costo c pesos cada uno" y $\text{Vender } n \ c$, "vender n dólares con la cotización c ".

La siguiente función calcula el saldo en pesos y dólares luego de una transacción:

```
eval :: Transaccion → (Pesos, Dolares)
eval (Depositar pesos) = (pesos, 0)
eval (Comprar n cot) = let x = fromIntegral n in (-x * cot, x)
eval (Vender n cot) = let x = fromIntegral n in (x * cot, -x)
```

Diremos que una secuencia de transacciones es correcta si al evaluar cada transacción en el orden en que ocurren en la secuencia todas pueden realizarse. Por ejemplo, transOk es correcta, mientras que transFail1 y transFail2 no, ya que la 2ª transacción no puede realizarse en transFail1 (no pueden venderse dólares sin haberlos comprado antes) ni en transFail2 (no se pueden comprar dólares sin un saldo suficiente en pesos).

```
transOk = [Depositar 100000, Comprar 100 1000, Vender 20 500, Comprar 5 1000]
transFail1 = [Depositar 100000, Vender 20 500, Comprar 100 1000, Comprar 5 1000]
transFail2 = [Depositar 100, Comprar 100 1000]
```

Usando las operaciones del TAD Secuencias, definir una función transaccionOk , que dada una secuencia de transacciones, determine si es correcta. Definir esta función con profundidad en $O(\lg n)$ y trabajo en $O(n)$, siendo n el largo de la secuencia.

2. Decimos que una secuencia es una progresión aritmética si la diferencia entre cualquier elemento (a excepción del primero) y su anterior es un valor constante. Por ejemplo, $\langle 3, 5, 7, 9 \rangle$ es una progresión aritmética mientras que $\langle 3, 5, 6, 8, 9 \rangle$ no lo es. Se quiere definir una función $\text{spaml} : \text{Seq Int} \rightarrow \text{Int}$ que dada una secuencia de enteros devuelve la longitud de la subsecuencia contigua más larga que es progresión aritmética.

Por ejemplo,

```
spaml(7, 6, 5, 4, 6, 8) = 4
spaml(6, 7, 5, 6) = 2
spaml(1, 2, 5, 8, 12, 14, 16) = 4 = 3
```

Dada una secuencia s de longitud n mayor a 2, una manera de calcular la longitud de la subsecuencia más larga que es progresión aritmética consiste en calcular para cada i mayor o igual a 1 y menor o igual que n el sufijo más largo que es progresión aritmética para la secuencia formada por los primeros i elementos de s .

Completar la siguiente definición de la función spaml , que utiliza las funciones del TAD secuencia, dando definiciones apropiadas para f , g y h , de manera que spaml tenga profundidad en $O(\lg n)$, donde n es el largo de la secuencia.

```
spaml : Seq Int → Int
spaml s | n ≤ 2    = n
        | otherwise = spaml_aux s
  where n = length s

spaml_aux : Seq Int → Int
spaml_aux s =
  let
    n      = length s
    s_dif  = tabulate (\i → nth (i + 1) s - nth i s) (n - 1)
    s_info = map f s_dif
    (s_red, r) = scan g (nth 0 s_info) (drop s_info 1)
    s_res   = map h (append s_red (singleton r))
    f = ...
    g = ...
    h = ...
  in 1 + reduce max 0 s_res
```

3. Usando las funciones del TAD secuencia, incluyendo necesariamente a la función scan , definir una función $\text{longestStreak} : \text{Float} \rightarrow \text{Seq Float} \rightarrow \text{Int}$, que dados

- un valor numérico val , que representa una temperatura
- una secuencia de valores numéricos s , que representa la temperatura máxima diaria a lo largo del tiempo

calcule la racha más larga de días en s donde la temperatura superó los val grados.

Por ejemplo,

```
longestStreak 30(20, 21, 27, 30, 24) = 0
longestStreak 30(28, 31, 32, 29, 31, 31, 33, 29) = 3
longestStreak 30(28, 31, 29, 31, 29) = 1
```

Definir longestStreak con profundidad en $O(\lg n)$, donde n es el largo de la secuencia.

2. Usando las operaciones del TAD Secuencia definir las siguientes funciones:

- a) $uniquify : Seq\ Int \rightarrow Seq\ Int$, que elimina los elementos duplicados de una secuencia de enteros. Esta función puede definirse con profundidad en $O((\lg n)^2)$, donde n es la longitud de la secuencia.
- b) $aa : Seq\ Char \rightarrow Int$, que dada una secuencia de caracteres cuenta la cantidad de subsecuencias contiguas "aa" que contiene. La subsecuencia "aaa" debe sumar 2. Por ejemplo,

$$aa\ \langle a, c, d, e, a, a, f, a, a, a \rangle = 3$$

La función puede definirse con profundidad en $O(\lg n)$, siendo n la longitud de la secuencia.